Definitions

Revision Guide

Every topic definition from the four study guides, step by step

- Part 1: C# Basics to Advanced
- Part 2: .NET Core and ASP.NET Core
- Part 3: React
- Part 4: C# Data Structures and Algorithms

Prepared by Kaif Shaikh • Quick Revision Edition

Table of Contents

Part 1: C# Basics to Advanced 4

1. Introduction to C# and .NET 4
2. Variables, Data Types and Operators 5
3. Control Flow: Decisions and Loops 5
4. Methods and Parameters 6
5. Strings and Arrays 7
6. Memory Management and Garbage Collection 7
7. Object-Oriented Programming (OOP) 8
8. Structs, Records, Enums, Tuples and Anonymous Types 10
9. Exception Handling 10
10. Collections (Complete Guide) 11
11. Generics 13
12. Delegates, Lambdas and Events 13
13. LINQ (Language Integrated Query) 14
14. Multithreading, Tasks and Async/Await 15
15. Advanced C# Features 16
16. C# Version History and Modern Features 17
17. Tricky Interview Questions and Output Puzzles 17
18. Quick Revision Cheat Sheet 18

Part 2: .NET Core and ASP.NET Core 19

1. Introduction to .NET Core and ASP.NET Core 19
2. Middleware and the Request Pipeline 20
3. Dependency Injection 20
4. Configuration, Options and Logging 21
5. Building Web APIs with Controllers 21
6. Minimal APIs 22
7. Filters and Error Handling 22
8. Entity Framework Core 23
9. Authentication, Authorization and Security 23
10. Caching and Performance 24
11. Background Services, HttpClient and Resilience 24
12. Real-Time, gRPC, Health Checks and More 25
13. Testing 25
14. Architecture and Design Patterns 26
15. Deployment 26
16. Interview Questions and Tricky Scenarios 27
17. Quick Revision Cheat Sheet 27

Part 3: React 28

1. Introduction to React 28
2. JavaScript Essentials for React 29
3. JSX in Depth 30
4. Components and Props 30
5. State and useState 31
6. Events and Forms 32
7. Rendering, Re-rendering and Reconciliation 32
8. useEffect and Side Effects 33
9. Hooks in Depth 34
10. Context API and State Management 35
11. Routing with React Router 35
12. Data Fetching and APIs 36
13. Performance Optimization 37
14. Advanced Patterns 37
15. Styling, TypeScript and Testing 38
16. React 18/19, Server Components and Next.js 38
17. Tricky Interview Questions and Output Puzzles 39
18. Quick Revision Cheat Sheet 39

Part 4: C# Data Structures and Algorithms 41

1. Introduction to DSA 41
2. Complexity Analysis (Big-O) 41
3. Arrays and Strings 42
4. Hashing 43
5. Recursion and Backtracking 43
6. Searching and Binary Search 43
7. Sorting Algorithms 44
8. Linked Lists 44
9. Stacks and Queues 45
10. Trees and Binary Search Trees 45
11. Heaps and Priority Queues 46
12. Graphs 47
13. Tries, Union-Find and Range Query Structures 47
14. Greedy Algorithms and Intervals 48
15. Dynamic Programming 48
16. Bit Manipulation and Math 48
17. Patterns, Interview Strategy and Revision 49

Part 1: C# Basics to Advanced

149 definitions in 18 chapters. Numbers match the sections of the CSharp_Basics_to_Advanced_Complete_Guide.pdf guide.

1. Introduction to C# and .NET

1.1 How to Use This Guide

A study guide that explains each C# topic in the same order: a simple explanation, a code example, then the tricky points and interview tips that go with it.

1.2 What is C#?

C# is a modern, type-safe, object-oriented programming language from Microsoft. It runs on .NET and is used for web APIs, desktop apps, games (Unity), cloud services and more.

1.3 C# vs .NET vs CLR: Clearing the Confusion

C# is the language, **.NET** is the platform (runtime + base class libraries + tools), and the **CLR** (Common Language Runtime) is the engine inside .NET that loads, JIT-compiles and runs the code.

1.4 How C# Code Actually Runs

C# source is compiled by Roslyn into **IL** (Intermediate Language) inside an assembly (.dll/.exe). At run time the CLR's **JIT compiler** turns that IL into native machine code for the current CPU.

1.5 Your First Program

The smallest complete C# application: code that compiles into an executable and has an entry point (`Main` or top-level statements) where execution starts.

1.6 Program Structure, Namespaces and using

A C# program is organised into **namespaces** (logical groups of types) that contain classes and other types. The `using` directive imports a namespace so its types can be used without their full names.

1.7 Comments and XML Documentation

Comments are notes the compiler ignores (`//` and `/* */`). **XML documentation comments** (`///`) describe APIs and feed IntelliSense and generated docs.

1.8 Console Input and Output

Reading input from and writing output to the terminal through the `System.Console` class (`ReadLine`, `Write`, `WriteLine`).

2. Variables, Data Types and Operators

2.1 Variables

A **variable** is a named storage location that holds a value of a specific type. C# is statically typed, so the type is fixed at compile time (explicitly, or inferred with `var`).

2.2 Built-in Data Types

The predefined types C# provides as keywords (`int`, `double`, `decimal`, `bool`, `char`, `string`, `object`...). Each keyword is an alias for a .NET type, e.g. `int` = `System.Int32`.

2.3 Value Types vs Reference Types

A **value type** variable holds its data directly, so assignment copies the data. A **reference type** variable holds a reference to an object on the heap, so assignment copies only the reference.

2.4 Constants: const vs readonly

`const` is a compile-time constant whose value is baked into the code. `readonly` is a field that can be assigned only at declaration or in a constructor, so its value is fixed at run time.

2.5 Type Conversion

Type conversion changes a value from one type to another: **implicit** (safe, automatic), **explicit** (a cast that may lose data), or through helpers such as `Convert`, `Parse` and `TryParse`.

2.6 Boxing and Unboxing

Boxing wraps a value type in an object on the heap so it can be treated as `object`. **Unboxing** extracts the value back with an explicit cast. Both cost time and memory.

2.7 Nullable Types and Null Operators

A **nullable value type** (`int?` = `Nullable<int>`) can hold a value or `null`. Null operators (`?.`, `??`, `??=`, `!`) work safely with values that may be null.

2.8 Operators

Operators are symbols that perform operations on operands: arithmetic, comparison, logical, bitwise, assignment, conditional (`?:`) and type operators (`is`, `as`, `typeof`).

2.9 Default Values

The value a variable or field gets when it is not explicitly assigned: `0` for numbers, `false` for bool, `null` for reference types. Available in code as `default(T)`.

3. Control Flow: Decisions and Loops

3.1 if / else if / else

A **conditional statement** that runs a block of code only when a boolean condition is true, with optional `else if` and `else` branches for the other cases.

3.2 switch Statement

A **multi-way branch** that compares one value against several case labels and runs the matching section. C# does not allow implicit fall-through between non-empty cases.

3.3 switch Expression (C# 8+)

A compact, **expression-based** form of switch that *returns a value*, using `pattern => result` arms and _ as the catch-all (discard) arm.

3.4 Loops

Loops repeat a block of code: `for` (counter-based), `while` (checks before each pass), `do-while` (runs at least once) and `foreach` (walks over a collection).

3.5 Jump Statements

Statements that move control elsewhere: `break` (leave a loop or switch), `continue` (skip to the next iteration), `return` (leave a method) and `goto` (jump to a label).

3.6 Pattern Matching Quick Reference

Pattern matching tests a value's shape and extracts data in one step, using type, constant, relational, logical, property, positional and list patterns with `is` and `switch`.

4. Methods and Parameters

4.1 Defining Methods

A **method** is a named block of code that performs a task. It has an access modifier, return type, name and parameter list, and is called to reuse logic.

4.2 Parameter Passing: By Value vs By Reference

By value (the default) passes a copy of the argument. **By reference** (`ref`, `out`, `in`) passes the variable itself, so the method can read or change the caller's variable.

4.3 Optional and Named Parameters

Optional parameters have default values, so callers may omit them. **Named arguments** pass values by parameter name (`age: 30`) instead of position.

4.4 params - Variable Number of Arguments

The `params` modifier lets a method accept a **variable number of arguments**, which the compiler packs into an array (or, from C# 13, another collection type).

4.5 Method Overloading (Compile-time Polymorphism)

Method overloading means defining several methods with the same name but different parameter lists. The compiler chooses which one to call at compile time.

4.6 Recursion

Recursion is when a method calls itself to solve a smaller instance of the same problem. It needs a base case that stops the calls.

4.7 Local Functions and Static Local Functions

A **local function** is a method declared inside another method and visible only there. A `static` local function cannot capture the enclosing method's variables.

4.8 Return Values: Tuples and Multiple Results

Ways to return more than one value from a method: **tuples** `(int, string)`, `out` parameters, or a custom class, record or struct.

5. Strings and Arrays

5.1 String Basics and Immutability

A `string` is an **immutable** sequence of UTF-16 characters (a reference type). Any "change" creates a new string object; the original never changes.

5.2 Creating Strings

Ways to build string values: literals, **verbatim** strings (`@"..."`), **interpolated** strings (`$"..."`), **raw** string literals (`"""..."""`), concatenation and `string.Join`/`Format`.

5.3 Common String Methods

The built-in methods of the `String` class for searching, comparing, slicing, splitting, trimming, padding and changing case. All of them return new strings.

5.4 StringBuilder

`StringBuilder` is a **mutable** character buffer for building a string step by step (for example in loops) without creating a new string on every change.

5.5 Arrays

An **array** is a fixed-size, zero-indexed collection of elements of one type, stored contiguously. C# has single-dimensional, multi-dimensional `[,]` and jagged `[][]` arrays.

5.6 Span<T> and Memory<T> (High-Performance Slicing)

Span<T> is a stack-only, type-safe view over a contiguous region of memory that lets you slice without copying. `Memory`<T> is its heap-storable counterpart that can be used in async code.

6. Memory Management and Garbage Collection

6.1 Stack vs Heap

The **stack** is a fast, per-thread LIFO memory area for method calls and local values, freed automatically when a method returns. The **heap** is a shared memory area for objects, freed by the garbage collector.

6.2 How the Garbage Collector Works

The **Garbage Collector (GC)** automatically frees heap memory used by objects that are no longer reachable. It uses generations (0, 1, 2) and the Large Object Heap to do this efficiently.

6.3 IDisposable and the using Statement

`IDisposable` is the pattern for **deterministically releasing unmanaged resources** (files, connections, handles). The `using` statement calls `Dispose()` automatically, even if an exception occurs.

6.4 Weak References

A `WeakReference` refers to an object **without keeping it alive**, so the GC can still collect it. Useful for caches.

7. Object-Oriented Programming (OOP)

7.1 Classes and Objects

A **class** is a blueprint that defines data (fields, properties) and behaviour (methods). An **object** is an instance of a class created with `new`, living on the heap.

7.2 The Four Pillars of OOP

The four core principles of object-oriented design: **Encapsulation** (hide data), **Abstraction** (expose only what is essential), **Inheritance** (reuse and extend) and **Polymorphism** (one interface, many forms).

7.3 Access Modifiers

Keywords that control **visibility** of types and members: `public`, `private`, `protected`, `internal`, `protected internal`, `private protected` and `file`.

7.4 Constructors

A **constructor** is a special method with the class name and no return type. It runs when an object is created to initialise its state. Variants: default, parameterised, static, private, chained and primary constructors.

7.5 Properties

A **property** is a member that looks like a field but uses `get`/`set` (or `init`) accessors, so access to the underlying data is controlled. Auto-properties generate the backing field for you.

7.6 static Members and Static Classes

`static` members belong to the **type itself**, not to any instance, and are shared by all callers. A static class holds only static members and cannot be instantiated.

7.7 Inheritance

Inheritance lets a class (derived) reuse and extend the members of another class (base) using `:`. C# supports single class inheritance and multiple interface implementation.

7.8 Polymorphism: virtual, override, new

Runtime polymorphism: a `virtual` method can be `override`-n in a derived class and the call is chosen by the object's real type. `new` *hides* the base member instead of overriding it.

7.9 Abstract Classes

An `abstract` class is an **incomplete base class** that cannot be instantiated. It can contain both implemented members and abstract members that derived classes must implement.

7.10 Interfaces

An **interface** is a **contract**: a set of members that implementing types promise to provide. A class can implement many interfaces. Default implementations are allowed from C# 8.

7.11 Abstract Class vs Interface

Use an **abstract class** for an "is-a" relationship with shared code and state (single inheritance). Use an **interface** for a "can-do" capability that many unrelated types can implement (multiple allowed).

7.12 System.Object Members

The members every type inherits from `System.Object`: `ToString`, `Equals`, `GetHashCode`, `GetType`, `MemberwiseClone` and `Finalize`.

7.13 Shallow Copy vs Deep Copy

A **shallow copy** copies an object's fields, so nested references are shared with the original. A **deep copy** also clones every referenced object, giving a fully independent copy.

7.14 Partial Classes and Partial Methods

`partial` lets one class, struct or interface be **split across multiple files** that the compiler merges. A partial method is declared in one part and implemented in another (common in generated code).

7.15 Nested Classes and the this Keyword

A **nested class** is a type declared inside another type, with access to its private members. `this` refers to the current instance (also used for constructor chaining, indexers and extension methods).

7.16 Operator Overloading and Conversion Operators

Operator overloading defines what operators such as + or == do for your own types. **Conversion operators** (`implicit`/`explicit`) define casts to and from them.

7.17 Indexers

An **indexer** lets objects of a class be accessed with array syntax (`obj[i]`). It is declared with `this[...]` and get/set accessors.

7.18 SOLID Principles (Quick Guide)

SOLID is five design principles for maintainable OO code: Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation and Dependency Inversion.

8. Structs, Records, Enums, Tuples and Anonymous Types

8.1 Structs

A `struct` is a lightweight **value type** for small data. It is copied on assignment, cannot inherit from another struct or class, and can implement interfaces.

8.2 Records (C# 9+)

A `record` is a type built for **immutable data**, with compiler-generated **value-based equality**, `ToString`, deconstruction and non-destructive mutation using `with`.

8.3 Enums

An `enum` is a value type that defines a set of **named integral constants**, making code readable. `[Flags]` enums combine values with bitwise operators.

8.4 Tuples

A **tuple** (`ValueTuple`) is a lightweight value type that groups several values, optionally named, without declaring a class: `(int Id, string Name)`.

8.5 Anonymous Types

An **anonymous type** is a read-only, compiler-generated class created inline with `new { Name = "A", Age = 3 }`. Mostly used for LINQ projections within one method.

8.6 Deconstruction for Your Own Types

Deconstruction unpacks an object into separate variables: `var (x, y) = point;`. Custom types support it by defining a `Deconstruct` method with `out` parameters.

9. Exception Handling

9.1 What is an Exception?

An **exception** is an object that represents an error raised at run time. It interrupts normal flow and travels up the call stack until a matching handler catches it.

9.2 try, catch, finally

`try` wraps code that may throw, `catch` handles specific exception types, and `finally` always runs for cleanup, whether or not an exception occurred.

9.3 throw vs throw ex

`throw;` re-throws the current exception and **keeps the original stack trace**. `throw ex;` resets the stack trace to the current line, which loses debugging information.

9.4 Tricky Cases with finally

Edge cases of `finally`: it runs even after `return`, it cannot change an already-evaluated return value, and it is skipped only in extreme cases such as process termination.

9.5 Exception Filters (when)

A **filter** (`catch (Ex e) when (condition)`) catches an exception only if the condition is true. Otherwise the stack is not unwound and the search for a handler continues.

9.6 Custom Exceptions

A **custom exception** is your own class derived from `Exception`. It represents a domain-specific error and carries extra details.

9.7 Common Built-in Exceptions

Standard exception types in .NET, such as `ArgumentNullException`, `InvalidOperationException`, `NullReferenceException`, `IndexOutOfRangeException` and `KeyNotFoundException`.

9.8 Guard Clauses (modern helpers)

Guard clauses validate inputs at the start of a method and fail fast. Modern helpers such as `ArgumentNullException.ThrowIfNull` and `ArgumentException.ThrowIfNullOrEmpty` do this in one line.

9.9 Best Practices

Proven rules for exception handling: catch specific exceptions, don't use exceptions for control flow, never swallow errors silently, preserve stack traces, and clean up with `using`/`finally`.

10. Collections (Complete Guide)

10.1 Overview and Hierarchy

Collections are classes that store and manage groups of objects. They are built on interfaces such as `IEnumerable<T>`, `ICollection<T>`, `IList<T>` and `IDictionary<K,V>` in `System.Collections.Generic`.

10.2 List<T>

`List<T>` is a **dynamic array**: an ordered, index-based collection that grows automatically. Index access and adding at the end are fast (O(1)).

10.3 Dictionary<TKey, TValue>

`Dictionary<TKey,TValue>` is a **hash table** of unique keys mapped to values, with near O(1) lookup, insert and delete by key.

10.4 HashSet<T>

`HashSet<T>` is an unordered collection of **unique** elements, with O(1) membership checks and set operations (union, intersect, except).

10.5 SortedSet<T>, SortedDictionary<K,V>, SortedList<K,V>

Sorted collections keep elements ordered by key or value automatically. `SortedSet` and `SortedDictionary` use red-black trees; `SortedList` uses sorted arrays.

10.6 Queue<T> - FIFO (First In, First Out)

Queue<T> is a **First-In-First-Out** collection: Enqueue adds at the back, and Dequeue removes from the front.

10.7 Stack<T> - LIFO (Last In, First Out)

Stack<T> is a **Last-In-First-Out** collection: Push adds to the top, and Pop removes from the top.

10.8 LinkedList<T>

`LinkedList<T>` is a **doubly-linked list** of nodes. It gives O(1) insert and remove at a known node, but O(n) access by position.

10.9 PriorityQueue<TElement, TPriority> (.NET 6)

`PriorityQueue<TElement,TPriority>` is a **min-heap**: Dequeue always returns the element with the lowest priority value.

10.10 Non-Generic Collections (Legacy)

The pre-generics collections (`ArrayList`, `Hashtable`...) in `System.Collections`. They store `object`, so they are not type-safe and box value types. Avoid them in new code.

10.11 Concurrent (Thread-Safe) Collections

Collections in `System.Collections.Concurrent` (`ConcurrentDictionary`, `ConcurrentQueue`, `BlockingCollection`...) that multiple threads can safely use at once without external locks.

10.12 Immutable, ReadOnly and Frozen Collections

ReadOnly collections are read-only *views* over data that can still change underneath. **Immutable** collections can never change. **Frozen** collections (.NET 8) are immutable and optimised for fast reads.

10.13 Big-O Complexity Cheat Sheet

Big-O notation describes how the cost of an operation grows with collection size, for example O(1) constant, O(log n) logarithmic and O(n) linear. Use this table to compare collections.

10.14 Which Collection Should I Use?

A decision guide that matches each need (order, uniqueness, key lookup, FIFO/LIFO, priority, thread safety) to the most suitable collection type.

10.15 Iterators and yield return

An **iterator** is a method that uses `yield return` to produce a sequence **lazily**, one element at a time. The compiler generates the `IEnumerator` state machine.

10.16 Custom Comparers and Equality

`IComparer<T>` defines custom **ordering**, and `IEqualityComparer<T>` defines custom **equality and hashing**. Sorts, sets and dictionaries use them in place of the default behaviour.

11. Generics

11.1 Why Generics?

Generics let you write classes and methods with **type parameters** (<T>), so one piece of code works for any type while staying type-safe and avoiding boxing.

11.2 Generic Methods

A **generic method** declares its own type parameter (`void Swap<T>(ref T a, ref T b)`). The compiler usually infers T from the arguments.

11.3 Generic Constraints

Constraints (`where T : ...`) limit which types can be used for T, such as `class`, `struct`, `new()`, `notnull`, a base class or an interface, so the code can use those members.

11.4 default(T) and Generic Math

`default(T)` gives the default value of any type parameter. **Generic math** (.NET 7, static abstract interface members such as `INumber<T>`) allows arithmetic on T.

11.5 Covariance and Contravariance

Variance allows implicit conversion of generic interfaces and delegates. **Covariance** (`out T`) uses a more derived type; **contravariance** (`in T`) uses a less derived type.

11.6 Static Fields in Generic Types

A static field in a generic type is **separate for each closed type**: `Cache<int>.Count` and `Cache<string>.Count` are different variables.

12. Delegates, Lambdas and Events

12.1 Delegates

A **delegate** is a **type-safe reference to a method** (a function pointer) with a specific signature, so methods can be passed around and invoked later.

12.2 Multicast Delegates

A **multicast delegate** holds an invocation list of several methods, added with += and removed with -=, and calls them in order. Only the last method's return value is kept.

12.3 Built-in Delegates: Func, Action, Predicate

Ready-made generic delegate types: `Func<..., TResult>` returns a value, `Action<...>` returns void, and `Predicate<T>` returns bool.

12.4 Anonymous Methods and Lambda Expressions

Inline, unnamed functions: **anonymous methods** (`delegate(int x) { ... }`) and the shorter **lambda expressions** (`x => x * 2`).

12.5 Closures and Captured Variables

A **closure** is a lambda that **captures variables** from its enclosing scope. It keeps a reference to the variable itself, not a copy of its value, so it extends the variable's lifetime.

12.6 Events

An **event** is a delegate wrapped by the `event` keyword so outsiders can only subscribe (`+=`) or unsubscribe (`-=`). It implements the publisher-subscriber pattern.

12.7 Expression Trees

An **expression tree** (`Expression<Func<...>>`) represents code as a **data structure** that can be inspected and translated, for example into SQL by Entity Framework.

13. LINQ (Language Integrated Query)

13.1 What is LINQ?

LINQ (Language Integrated Query) is a set of query operators and syntax for filtering, projecting, sorting and grouping data from any source (collections, databases, XML) directly in C#.

13.2 Deferred vs Immediate Execution

Deferred execution means a query runs only when it is enumerated, and runs again each time. **Immediate** execution (`ToList`, `Count`, `First`...) runs it at once and stores the result.

13.3 Filtering and Projection

Filtering (`Where`, `OfType`) selects the elements that match a condition. **Projection** (`Select`, `SelectMany`) transforms each element into a new shape.

13.4 Element Operators: First, Single, Last, ElementAt

Operators that return a **single element**: `First`, `Single`, `Last`, `ElementAt`. They differ in how they treat zero or multiple matches; the `...OrDefault` versions return a default value instead of throwing.

13.5 Ordering

Sorting a sequence with `OrderBy`/`OrderByDescending`, then adding secondary keys with `ThenBy`/`ThenByDescending`. LINQ ordering is stable.

13.6 Aggregation

Operators that **reduce a sequence to one value**: `Count`, `Sum`, `Min`, `Max`, `Average`, `MinBy`/`MaxBy` and the general-purpose `Aggregate`.

13.7 Quantifiers

Operators that return a **bool** about a sequence: `Any` (at least one matches), `All` (every element matches) and `Contains` (a value is present).

13.8 Grouping

`GroupBy` splits a sequence into **groups that share a key**; each group is an `IGrouping<TKey,T>`. `ToLookup` does the same eagerly.

13.9 Joins

Joins combine two sequences by matching keys: `Join` (inner join), `GroupJoin` (for left outer joins) and `Zip` (pairs elements by position).

13.10 Set Operators

Operators that treat sequences as **mathematical sets**: `Distinct`, `Union`, `Intersect`, `Except`, plus the key-based `...By` variants.

13.11 Partitioning, Paging and Others

Partitioning operators take or skip part of a sequence (`Take`, `Skip`, `TakeWhile`, `Chunk`). They are the building blocks for **paging** results.

13.12 IEnumerable vs IQueryable

`IEnumerable<T>` runs queries **in memory** using delegates. `IQueryable<T>` builds expression trees that a provider translates and runs **at the data source**, such as SQL.

13.13 PLINQ (Parallel LINQ)

PLINQ (`AsParallel()`) runs LINQ queries in **parallel** across multiple CPU cores to speed up CPU-bound work on large data sets.

14. Multithreading, Tasks and Async/Await

14.1 Key Concepts

Core concurrency terms: a **process** is a running program, a **thread** is a unit of execution within it, **concurrency** means making progress on several tasks, and **parallelism** means running them at literally the same time.

14.2 Threads

A `Thread` is an OS-level **path of execution** you create and control directly. It is expensive to create, so modern code prefers tasks and the thread pool.

14.3 Tasks

A `Task` (or `Task<T>`) represents an **asynchronous operation** that will finish in the future. It usually runs on the thread pool and can be awaited, composed and cancelled.

14.4 async and await

`async` marks a method that can `await`. `await` pauses the method **without blocking the thread** until a task completes, then resumes. The compiler builds a state machine.

14.5 Sequential vs Concurrent Awaits

Sequential awaits run tasks one after another (total time = sum). **Concurrent** awaits start all tasks first and then `await Task.WhenAll` (total time = the slowest task).

14.6 Async Pitfalls and Tricky Points

Common async mistakes: deadlocks from `.Result`/`.Wait()`, `async void`, forgetting to `await`, misusing `Task.Run`, and eliding `async`/`await` unsafely.

14.7 Cancellation

Cooperative cancellation: a `CancellationTokenSource` signals a `CancellationToken` that long-running or async operations check so they can stop early.

14.8 Thread Synchronisation

Techniques that stop threads from corrupting shared data: `lock`/`Monitor`, `Interlocked`, `SemaphoreSlim`, `Mutex`, `ReaderWriterLockSlim` and the .NET 9 `Lock` type.

14.9 Parallel Class

The `Parallel` class (`Parallel.For`, `ForEach`, `ForEachAsync`, `Invoke`) splits **CPU-bound work across cores** automatically using the thread pool.

14.10 Async Streams and Channels

Async streams (`IAsyncEnumerable<T>` with `await foreach`) produce data asynchronously, item by item. **Channels** are thread-safe async queues for producer/consumer pipelines.

14.11 Thread-Safe Lazy Initialisation and Singleton

Creating an object **once, on first use, safely across threads**, for example with Lazy<T> or a static initialiser. This is the standard way to build a thread-safe Singleton.

15. Advanced C# Features

15.1 Extension Methods

An **extension method** is a static method in a static class whose first parameter uses `this`. It can then be called as if it were an instance method of that type, which you don't have to own.

15.2 Nullable Reference Types (C# 8+)

A compiler feature (`#nullable enable`) where reference types are **non-nullable by default** and `string?` marks the ones that may be null. The compiler then warns about possible null dereferences.

15.3 Attributes

An **attribute** is declarative **metadata** attached to code elements with `[Name]`. Compilers, frameworks or reflection read it at compile time or run time.

15.4 Reflection

Reflection (`System.Reflection`) lets a program **inspect and use type metadata at run time**: list members, create instances, invoke methods and read attributes dynamically.

15.5 typeof vs GetType vs is

`typeof(T)` gets a Type at **compile time** from a type name. `obj.GetType()` gets the **actual run-time** type of an instance. `is` checks compatibility, including inheritance.

15.6 dynamic

`dynamic` **bypasses compile-time type checking**. Member access is resolved at run time by the DLR, which gives flexibility (COM, JSON, interop) at the cost of safety and speed.

15.7 Unsafe Code and Pointers

`unsafe` code allows **pointers** and direct memory manipulation (`*`, `&`, `fixed`, `stackalloc`) outside the CLR's memory-safety checks. It requires the `AllowUnsafeBlocks` setting.

15.8 File I/O

Reading and writing files and directories through `System.IO`: `File`, `Directory`, `Path`, streams, `StreamReader`/`StreamWriter` and their async versions.

15.9 JSON Serialization (System.Text.Json)

Serialization converts objects to JSON text, and deserialization converts it back. `System.Text.Json` (`JsonSerializer`) is the fast, built-in .NET library for both.

15.10 Dependency Injection

Dependency Injection means a class **receives its dependencies** (usually through its constructor) instead of creating them. A DI container wires them up with Transient, Scoped or Singleton lifetimes.

15.11 Common Design Patterns in C#

Design patterns are reusable solutions to common design problems, for example Singleton, Factory, Builder, Strategy, Observer, Decorator, Repository and Adapter.

15.12 Preprocessor Directives and Conditional Compilation

Preprocessor directives (`#if`, `#define`, `#region`, `#pragma`...) are instructions processed before compilation, used to include or exclude code by build symbol such as `DEBUG`.

15.13 Date, Time and Numeric Formatting

Working with dates and times (`DateTime`, `DateTimeOffset`, `DateOnly`, `TimeSpan`) and turning values into text with **format strings** (`"N2"`, `"C"`, `"yyyy-MM-dd"`) and cultures.

16. C# Version History and Modern Features

16.1 Feature Timeline

A version-by-version list of the major language features C# has added, from C# 1.0 (2002) to C# 14 (.NET 10).

16.2 Modern Features in Action

Short, practical examples of recent C# features (records, pattern matching, primary constructors, collection expressions and more) used together in real code.

16.3 Top-Level Program for a Minimal API (.NET 8+)

A **Minimal API** is an ASP.NET Core web API written with top-level statements and `app.MapGet`/`MapPost`, without controllers or a `Startup` class.

17. Tricky Interview Questions and Output Puzzles

17.1 Output Prediction Puzzles

Short code snippets where you **predict the exact output**. They test a precise understanding of language rules such as closures, value vs reference types, statics and evaluation order.

17.2 Frequently Asked Conceptual Questions

The theory questions interviewers ask most often, each with a concise answer you can give in one or two sentences.

17.3 Coding Questions to Practise

Common hands-on coding problems (strings, arrays, collections, LINQ) with sample solutions, for practising clean, correct C# under interview conditions.

18. Quick Revision Cheat Sheet

18.1 Syntax at a Glance

A compact reference of everyday C# syntax: declarations, control flow, classes, collections, LINQ and async, in one place for quick lookup.

18.2 Top 25 Tricky Points (Revise Before Interviews)

The 25 most important "gotchas" from this guide, gathered for fast revision before an interview.

18.3 Suggested Study Roadmap

A suggested order of study, from fundamentals to advanced topics, with milestones that build practical C# skill step by step.

Part 2: .NET Core and ASP.NET Core

80 definitions in 17 chapters. Numbers match the sections of the DotNet_Core_Basics_to_Advanced_Complete_Guide.pdf guide.

1. Introduction to .NET Core and ASP.NET Core

1.1 How to Use This Guide

A study guide that explains each ASP.NET Core topic in the same order: a simple explanation, a working code example, then the tricky points and interview tips that go with it.

1.2 What is .NET Core?

.NET Core (now just **.NET**, version 5 and later) is Microsoft's free, open-source, cross-platform runtime and framework. **ASP.NET Core** is the part of it for building web apps and APIs.

1.3 .NET Architecture

The layers of .NET: the **runtime** (CLR: JIT, GC, type system), the **Base Class Library**, **app models** such as ASP.NET Core and EF Core, and the **SDK** tooling that builds and runs apps.

1.4 The dotnet CLI

The **dotnet CLI** is the cross-platform command-line tool for creating, building, running, testing, publishing and managing packages of .NET projects (`dotnet new`, `build`, `run`, `test`, `publish`).

1.5 Project Structure

The files and folders of an ASP.NET Core project: the `.csproj` project file, `Program.cs` (entry point), `appsettings.json` (configuration), `Properties/launchSettings.json`, and folders such as Controllers and Models.

1.6 Program.cs: The Minimal Hosting Model (.NET 6+)

The **minimal hosting model** configures and starts the whole app in `Program.cs`: `WebApplication.CreateBuilder` registers services, then `builder.Build()` creates the app where middleware and endpoints are added before `app.Run()`.

1.7 Hosting: Kestrel, IIS and Reverse Proxies

Kestrel is ASP.NET Core's built-in, cross-platform web server. In production it often runs behind a **reverse proxy** (IIS, Nginx, a load balancer) that forwards requests to it and handles TLS and routing.

1.8 Generic Host and the App Lifecycle

The **Generic Host** (`IHost`) owns the app's DI container, configuration, logging and hosted services, and manages its **lifecycle**: start, run, and graceful shutdown (`IHostApplicationLifetime`).

2. Middleware and the Request Pipeline

2.1 What is Middleware?

Middleware is a component in the request pipeline. Each one can handle an HTTP request, pass it to the next component, and act on the response on its way back.

2.2 Use, Run and Map

Ways to add middleware: `Use` adds a component that can call the next one, `Run` adds a **terminal** component that ends the pipeline, and `Map`/`MapWhen` branch the pipeline by path or condition.

2.3 Writing a Custom Middleware Class

A **middleware class** takes a `RequestDelegate next` in its constructor and has an `InvokeAsync(HttpContext)` method. It is registered with `app.UseMiddleware<T>()`.

2.4 Recommended Middleware Order

The order in which middleware must be added, because the pipeline runs in registration order: exception handling, HTTPS, static files, routing, CORS, authentication, authorization, then endpoints.

2.5 Built-in Middleware Overview

The ready-made middleware ASP.NET Core provides, such as exception handling, HTTPS redirection, static files, routing, CORS, authentication, authorization, response compression, rate limiting and output caching.

3. Dependency Injection

3.1 Why Dependency Injection?

Dependency Injection means a class receives the services it needs (usually through its constructor) instead of creating them. This gives loose coupling, easier testing and central configuration.

3.2 Service Lifetimes

How long a registered service instance lives: **Transient** (new every time it's requested), **Scoped** (one per HTTP request) and **Singleton** (one for the whole app).

3.3 Resolving Services

Ways to get services from the container: **constructor injection** (preferred), `[FromServices]` or Minimal API parameters, and `IServiceProvider`/`CreateScope()` when a scope is needed manually.

3.4 Multiple Implementations and Keyed Services

Registering several implementations of one interface: inject `IEnumerable<T>` to get all of them, or use **keyed services** (.NET 8) with `[FromKeyedServices("key")]` to pick one by name.

3.5 Disposal and Open Generics

The container **disposes** the `IDisposable` services it created when their scope ends. **Open generics** register one generic type for every type argument, e.g. `IRepository<>` with `Repository<>`.

3.6 Third-Party Containers and Scrutor

Replacement or add-on DI containers (Autofac and others) for advanced features, and **Scrutor**, a library that adds assembly scanning and decoration to the built-in container.

4. Configuration, Options and Logging

4.1 Configuration Sources

Configuration is a key-value system built from layered **sources**, where later sources override earlier ones: `appsettings.json`, `appsettings.{Environment}.json`, user secrets, environment variables, command-line args and Key Vault.

4.2 The Options Pattern

The **Options pattern** binds a configuration section to a strongly typed class, which is injected as `IOptions<T>`, `IOptionsSnapshot<T>` or `IOptionsMonitor<T>` and can be validated at startup.

4.3 Logging with ILogger

`ILogger<T>` is the built-in logging abstraction. It writes **structured** log messages at levels (Trace to Critical) to configured providers such as Console, Debug or Serilog.

5. Building Web APIs with Controllers

5.1 REST Basics

REST is an architectural style for APIs: resources are identified by URLs, handled with standard HTTP verbs (GET, POST, PUT, PATCH, DELETE), are stateless, and return meaningful status codes.

5.2 A Complete Controller

A **controller** is a class (usually inheriting `ControllerBase` with `[ApiController]`) whose public **action methods** handle HTTP requests for one resource.

5.3 Routing

Routing maps an incoming URL and HTTP verb to an endpoint. Controllers use **attribute routing** (`[Route]`, `[HttpGet("{id:int}")]`) with route templates and constraints.

5.4 Model Binding

Model binding automatically converts request data (route values, query string, headers, form and JSON body) into action parameters. Sources can be set with `[FromRoute]`, `[FromQuery]` and `[FromBody]`.

5.5 Validation

Validation checks that incoming models are correct, using Data Annotations (`[Required]`, `[Range]`) or FluentValidation. With `[ApiController]`, invalid models return 400 automatically.

5.6 Action Results and Return Types

What an action returns: a specific type, `IActionResult`, or `ActionResult<T>`, created with helpers such as `Ok()`, `NotFound()`, `CreatedAtAction()` and `NoContent()` to set the status code.

5.7 DTOs and Mapping

DTOs (Data Transfer Objects) are simple classes or records shaped for the API contract, kept separate from database entities. **Mapping** converts between the two, by hand or with a library like Mapster.

5.8 API Versioning and OpenAPI

API versioning lets several versions of an API coexist (through the URL, query or a header). **OpenAPI** (Swagger) is a machine-readable description of the API, used for documentation and client generation.

5.9 JSON Serialization Settings

Settings for how `System.Text.Json` reads and writes request and response bodies: naming policy, enum handling, null handling, reference cycles and custom converters.

6. Minimal APIs

6.1 What are Minimal APIs?

Minimal APIs define HTTP endpoints directly with lambdas (`app.MapGet`, `MapPost`...) without controllers. They are lightweight and fast, and well suited to microservices.

6.2 Organising Endpoints: Route Groups and TypedResults

`MapGroup` creates **route groups** that share a prefix, filters and metadata. `TypedResults` return strongly typed results, which improve OpenAPI output and testability.

6.3 Endpoint Filters and Validation

Endpoint filters (`AddEndpointFilter`) run code before and after a Minimal API handler, for example to validate input or log, much like action filters do for controllers.

6.4 Controllers vs Minimal APIs

A comparison of the two API styles. Controllers suit large, convention-based APIs with filters and model-binding features. Minimal APIs suit small, fast, focused services.

7. Filters and Error Handling

7.1 MVC Filter Pipeline

Filters run code at specific stages of MVC action execution: Authorization, Resource, Action, Exception and Result filters, each wrapping the next stage.

7.2 Global Error Handling with ProblemDetails

Handling unhandled exceptions in one place (`UseExceptionHandler` or `IExceptionHandler`) and returning a standard **ProblemDetails** (RFC 7807/9457) JSON error response.

7.3 Exceptions vs Result Objects

Two ways to report failures: **throwing exceptions** for unexpected errors, or returning a **Result object** (success or error value) for expected business failures, without the cost of exceptions.

8. Entity Framework Core

8.1 What is EF Core?

Entity Framework Core is .NET's **ORM** (Object-Relational Mapper). It maps C# classes to database tables and lets you query and save data with LINQ through a `DbContext`.

8.2 Migrations (Code First)

Migrations keep the database schema in sync with the C# model (Code First): `dotnet ef migrations add` creates a schema change script, and `database update` applies it.

8.3 Querying

Reading data with LINQ against `DbSet<T>`, which EF Core translates to SQL. Key tools are filtering, projection, `AsNoTracking`, pagination, and raw SQL when needed.

8.4 Loading Related Data

Strategies for loading navigation properties: **eager** (`Include`), **explicit** (`Entry().Load`), **lazy** (proxies, loaded on access), and **split queries** to avoid cartesian explosion.

8.5 Saving Data and Change Tracking

The **change tracker** records the state of loaded entities (Added, Modified, Deleted...). `SaveChanges()` turns those changes into INSERT, UPDATE and DELETE statements in one transaction.

8.6 Transactions and Concurrency

Transactions group several operations so they succeed or fail together. **Concurrency control** (optimistic, using a row version or concurrency token) detects conflicting updates by different users.

8.7 DbContext Rules and Performance

Best practices for `DbContext`: it is **scoped and not thread-safe**. Use `AsNoTracking` for reads, project only the needed columns, avoid N+1 queries, and consider pooling and compiled queries.

9. Authentication, Authorization and Security

9.1 Authentication vs Authorization

Authentication establishes *who* the user is (identity). **Authorization** decides *what* that authenticated user is allowed to do (permissions).

9.2 JWT Bearer Authentication

JWT (JSON Web Token) is a signed token containing user claims. With **bearer authentication**, the client sends it in the `Authorization: Bearer` header and the API validates it on every request.

9.3 Authorization: Roles, Claims and Policies

Ways to control access: **role-based** (`[Authorize(Roles="Admin")]`), **claims-based**, and **policy-based** authorization, where named policies combine requirements and handlers.

9.4 ASP.NET Core Identity

ASP.NET Core Identity is a membership system for users, passwords, roles, claims, tokens, lockout and two-factor authentication, stored through EF Core.

9.5 CORS

CORS (Cross-Origin Resource Sharing) is a browser security mechanism. The server sends headers that say which other origins (domains) may call the API from JavaScript.

9.6 Rate Limiting (.NET 7+)

Rate limiting restricts how many requests a client may make in a time window (fixed window, sliding window, token bucket, concurrency) to protect the API from abuse. Built in via `AddRateLimiter`.

9.7 Security Checklist

A list of essential security practices: HTTPS and HSTS, input validation, parameterised queries, secret management, least privilege, security headers, and dependency updates.

10. Caching and Performance

10.1 Caching Options

Ways to store data so it can be served faster: **in-memory** (`IMemoryCache`), **distributed** (`IDistributedCache`, e.g. Redis), **HybridCache**, **output caching** and **response caching**.

10.2 Async All the Way

Using `async/await` through every layer for I/O work, so request threads are freed while waiting and the server can handle many more requests. Avoid `.Result/.Wait()`.

10.3 Performance Checklist

A list of techniques for fast APIs: async I/O, caching, efficient EF queries, pagination, response compression, pooling, avoiding large allocations, and profiling before optimising.

11. Background Services, HttpClient and Resilience

11.1 Hosted Services and BackgroundService

A **hosted service** (`IHostedService`) runs background work alongside the web app. `BackgroundService` is a base class for long-running loops, via ExecuteAsync.

11.2 IHttpClientFactory

`IHttpClientFactory` creates and manages `HttpClient` instances, pooling handlers to avoid socket exhaustion and stale DNS. It supports named and typed clients.

11.3 Resilience with Polly (Microsoft.Extensions.Http.Resilience)

Resilience means handling transient failures in calls to other services with strategies such as retry, timeout, circuit breaker and fallback, using Polly via **Microsoft.Extensions.Http.Resilience**.

12. Real-Time, gRPC, Health Checks and More

12.1 SignalR (Real-Time Communication)

SignalR is a library for **real-time**, two-way communication between server and clients (chat, notifications, live dashboards). It uses WebSockets with automatic fallbacks.

12.2 gRPC

gRPC is a high-performance RPC framework that uses HTTP/2 and Protocol Buffers (binary, contract-first `.proto` files). It suits fast service-to-service communication.

12.3 Health Checks

Health checks expose endpoints (e.g. `/health`) that report whether the app and its dependencies (database, cache) are healthy. Load balancers and orchestrators use them.

12.4 Other Useful Features

Other ASP.NET Core features you should know about, such as Razor Pages and MVC views, Blazor, output caching, response compression and more.

13. Testing

13.1 Testing Pyramid

The **testing pyramid**: many fast **unit tests** at the base, fewer **integration tests** in the middle, and a small number of slow **end-to-end tests** at the top.

13.2 Unit Testing with xUnit and Moq

Unit testing checks one class in isolation. **xUnit** is the test framework (Fact, Theory), and **Moq** creates fake (mock) dependencies so behaviour can be controlled and verified.

13.3 Integration Testing with WebApplicationFactory

Integration testing runs the real app in memory with `WebApplicationFactory<Program>` and sends real HTTP requests through the full pipeline, often with a test database.

14. Architecture and Design Patterns

14.1 Clean Architecture

Clean Architecture organises code into layers (Domain, Application, Infrastructure, Presentation) where dependencies point **inward**, so business rules don't depend on frameworks or databases.

14.2 Repository and Unit of Work

The **Repository** pattern hides data access behind a collection-like interface. **Unit of Work** groups repository changes into one transaction (EF Core's DbContext already implements both).

14.3 CQRS and MediatR

CQRS (Command Query Responsibility Segregation) separates writes (commands) from reads (queries). **MediatR** is a library that sends each request to its single handler (the mediator pattern).

14.4 Other Patterns You'll Meet

Other design patterns common in ASP.NET Core apps, such as Options, Decorator, Strategy, Factory, Specification and Result, and where each is used.

14.5 Microservices Basics

Microservices is an architecture where an application is split into small, independently deployable services, each owning its data and communicating over HTTP, gRPC or messaging.

15. Deployment

15.1 Publish Options

Ways to package an app with `dotnet publish`: **framework-dependent** (needs .NET installed) or **self-contained** (includes the runtime), with options such as single-file, trimming and Native AOT.

15.2 Hosting on IIS (Windows)

Running ASP.NET Core on Windows **IIS** through the **ASP.NET Core Module**, either in-process (inside the IIS worker process) or out-of-process (IIS as a reverse proxy to Kestrel).

15.3 Docker

Docker packages the app and its dependencies into a portable **container image**, built from a Dockerfile (usually multi-stage), so it runs the same way everywhere.

15.4 Cloud and CI/CD

Hosting on cloud platforms (Azure App Service, containers, Kubernetes) and **CI/CD**: automated pipelines (GitHub Actions, Azure DevOps) that build, test and deploy on every change.

15.5 Production Readiness Checklist

Final checks before going live: configuration and secrets, HTTPS, logging and monitoring, health checks, migrations, resilience, scaling and security hardening.

16. Interview Questions and Tricky Scenarios

16.1 Scenario-Based Puzzles

Realistic "what goes wrong here?" scenarios, such as wrong middleware order or captive dependencies, each with the cause and the fix. They test practical debugging skill.

16.2 Frequently Asked Questions

The ASP.NET Core theory questions interviewers ask most often, each with a short, precise answer.

17. Quick Revision Cheat Sheet

17.1 Program.cs Template

A ready-to-use `Program.cs` for a production-style Web API, with services, middleware and endpoints wired in the correct order.

17.2 Top 25 Tricky Points

The 25 most important ASP.NET Core "gotchas" from this guide, gathered for fast revision before an interview.

17.3 Study Roadmap

A suggested order of study, from fundamentals to advanced topics and deployment, with practice tasks for each stage.

Part 3: React

123 definitions in 18 chapters. Numbers match the sections of the ReactJS_Basics_to_Advanced_Complete_Guide.pdf guide.

1. Introduction to React

1.1 How to Use This Guide

A study guide that explains each React topic in the same order: a simple explanation, a code example, then the tricky points and interview tips that go with it.

1.2 What is React?

React is an open-source JavaScript **library** from Meta for building user interfaces out of reusable **components**. It updates the page efficiently when data (state) changes.

1.3 Library vs Framework, and the React Ecosystem

A **library** solves one job and you call it (React handles the UI). A **framework** controls the whole app structure and calls your code (Angular, Next.js). The React **ecosystem** adds routing, state, data-fetching and build tools around it.

1.4 SPA, MPA and SSR

SPA (Single-Page App): one HTML page, and JavaScript renders every view in the browser. **MPA**: the server sends a new page for each navigation. **SSR**: the server renders HTML for each request, which the client then makes interactive.

1.5 How React Works Under the Hood

React keeps a lightweight **Virtual DOM** description of the UI. On each update it **reconciles** (diffs) the new description with the previous one and applies only the needed changes to the real DOM. The **Fiber** engine schedules this work.

1.6 Setting Up a Project

Creating a new React project with a build tool such as **Vite** (or a framework like Next.js), which provides a dev server, JSX compilation, hot reload and production bundling.

1.7 Your First React App

A minimal React application: a root component rendered into a DOM element with `createRoot(...).render(<App />)`.

1.8 React Versions at a Glance

A timeline of major React releases and what each added, from JSX and the Virtual DOM to Hooks (16.8), concurrent rendering (18) and Actions and Server Components (19).

2. JavaScript Essentials for React

2.1 let, const and var

Ways to declare variables: `let` (block-scoped, can be reassigned), `const` (block-scoped, cannot be reassigned) and the legacy `var` (function-scoped and hoisted).

2.2 Arrow Functions

A short function syntax (`(a, b) => a + b`) with implicit return for single expressions. Arrow functions do **not** have their own `this`; they use the surrounding one.

2.3 Destructuring

Destructuring unpacks values from arrays or properties from objects into variables in one statement: `const { name, age } = user`, `const [a, b] = arr`.

2.4 Spread and Rest

The `...` operator. **Spread** expands an array or object into elements or properties (used for copying and merging). **Rest** collects the remaining elements or properties into one variable.

2.5 Template Literals, Optional Chaining, Nullish Coalescing

Template literals (`` `Hi ${name}` ``) build strings with embedded expressions. **Optional chaining** (`a?.b`) safely reads properties that may be missing. **Nullish coalescing** (`a ?? b`) gives a default only for null or undefined.

2.6 Truthy and Falsy

In a boolean context, JavaScript treats some values as **falsy** (`false`, `0`, `''`, `null`, `undefined`, `NaN`) and everything else as **truthy**. This matters for conditional rendering.

2.7 Array Methods You Will Use Daily

The array methods used constantly in React for rendering and immutable updates: `map`, `filter`, `find`, `reduce`, `some`, `includes`, `slice` and `toSorted`.

2.8 Equality and References

=== compares primitives by value but objects and arrays by **reference**, meaning whether they are the same object in memory. React uses this (Object.is) to decide whether state or props changed.

2.9 Modules: import and export

ES modules split code across files: `export` makes values available to other files, and `import` brings them in. There are default exports and named exports.

2.10 Closures

A **closure** is a function that remembers the variables of the scope where it was created, even after that scope has finished. Every event handler and hook callback is a closure.

2.11 Promises and async/await

A **Promise** represents a value that will be available later (pending, then fulfilled or rejected). `async/await` is syntax for writing promise-based code that reads like synchronous code.

2.12 The Event Loop in One Minute

The **event loop** is how single-threaded JavaScript handles async work. It runs the call stack, then all **microtasks** (promises), then the next **macrotask** (timers, events).

3. JSX in Depth

3.1 What is JSX?

JSX is a syntax extension that lets you write HTML-like markup inside JavaScript. It compiles to `React.createElement` (or `jsx()`) calls that describe the UI.

3.2 JSX Rules

The rules JSX enforces: return a single root element, close every tag, use `className` and camelCase attributes, and put JavaScript expressions inside `{ }`.

3.3 Fragments

A **Fragment** (`<>...</>` or `<Fragment>`) groups several elements without adding an extra DOM node.

3.4 Conditional Rendering

Showing different UI depending on a condition, using `if`, the ternary `? :`, or `&&` inside JSX (watch out for `0` being rendered).

3.5 Rendering Lists and Keys

Rendering arrays with `map()`. Each item needs a stable, unique **key** prop so React can track which items were added, removed or reordered.

3.6 Rendering HTML Strings and Escaping

React **escapes** all values rendered in JSX to prevent XSS. To insert raw HTML you must use `dangerouslySetInnerHTML`, and the HTML should be sanitised first.

4. Components and Props

4.1 Function Components

A **function component** is a JavaScript function, named in PascalCase, that receives props and returns JSX describing part of the UI. It is the modern way to write React components.

4.2 Props

Props (properties) are the **read-only** inputs a parent passes to a child component, like function arguments. They configure the child's output.

4.3 The children Prop and Composition

`children` is a special prop containing whatever is placed between a component's tags. **Composition** builds complex UIs by nesting components rather than through inheritance.

4.4 One-Way Data Flow

In React, data flows **downward** from parent to child through props. Children communicate upward by calling callback functions that the parent passes as props.

4.5 Pure Components

A **pure component** always returns the same JSX for the same props and state, and has no side effects during rendering. React relies on this for correctness and optimisation.

4.6 Prop Validation: TypeScript and PropTypes

Checking that components receive props of the right type: **TypeScript** types at compile time (preferred), or the legacy run-time **PropTypes** library.

4.7 Component Design Tips

Guidelines for good components: keep them small and single-purpose, name them clearly, lift state only as high as needed, and prefer composition and clear prop interfaces.

5. State and useState

5.1 What is State?

State is data a component remembers between renders that changes over time. Updating it with the setter from `useState` causes the component to re-render.

5.2 State is a Snapshot

During a render, a state variable is a fixed **snapshot**. Calling the setter does not change the current value; it schedules a re-render that uses the new value.

5.3 Batching

Batching means React groups several state updates made in the same event (in React 18+, everywhere) into a single re-render, for better performance.

5.4 Lazy Initial State

Passing a **function** to `useState(() => expensive())` so the expensive initial value is computed only on the first render, not on every render.

5.5 Updating Objects Immutably

Never mutate state objects. Create a **new object** with the changes (`{ ...obj, key: value }`) so React sees a new reference and re-renders.

5.6 Updating Arrays Immutably

Updating state arrays by creating **new arrays**: `[...arr, x]` to add, `filter` to remove, `map` to update, `toSorted` to sort. Avoid `push`, `splice` and in-place `sort`.

5.7 Lifting State Up

Lifting state up means moving shared state to the closest common parent of the components that need it, then passing it down as props along with updater callbacks.

5.8 Derived State: Don't Store What You Can Calculate

Derived state is a value that can be calculated from existing props or state. Compute it during render instead of storing it in separate state, which can get out of sync.

5.9 State Preservation and Position in the Tree

React keeps a component's state as long as the **same component type** is rendered at the **same position** in the tree. Changing the type or the key resets it.

6. Events and Forms

6.1 Handling Events

Responding to user actions by passing a **function** to camelCase event props such as `onClick` and `onChange`: `onClick={handleClick}`, not `handleClick()`.

6.2 Synthetic Events

A **SyntheticEvent** is React's cross-browser wrapper around the native DOM event, with the same interface (preventDefault, stopPropagation, target).

6.3 Controlled Components

A **controlled component** is a form input whose value is driven by React state (`value` + `onChange`). React state is the single source of truth.

6.4 Uncontrolled Components

An **uncontrolled component** keeps its own value in the DOM. React reads it when needed through a `ref` or FormData, and sets only the `defaultValue`.

6.5 React 19 Form Actions

In React 19, a **form action** is a function passed to `<form action={fn}>`. React handles submission, pending state (`useFormStatus`) and results (`useActionState`) for you.

6.6 Form Libraries

Libraries that simplify complex forms: **React Hook Form** (fast, uncontrolled-input based) together with a schema validator such as **Zod** for validation rules.

7. Rendering, Re-rendering and Reconciliation

7.1 Trigger, Render, Commit

The three steps of every UI update: a **trigger** (initial render or state change), the **render** phase (React calls components to compute the new UI) and the **commit** phase (React applies changes to the DOM).

7.2 When Does a Component Re-render?

A component re-renders when its **state** changes, its **parent** re-renders, or a **context** it uses changes. Prop changes alone are not the trigger; the parent's re-render is.

7.3 Reconciliation and the Diffing Algorithm

Reconciliation is how React compares the new element tree with the previous one. Its O(n) **diffing** algorithm assumes different element types produce different trees and uses keys to match list items.

7.4 Mounting, Updating, Unmounting

The lifecycle of a component: **mounting** (first added to the screen), **updating** (re-rendering with new props or state) and **unmounting** (removed from the screen).

7.5 Class Components and Lifecycle Methods (Legacy)

The older way to write components, as ES6 classes with `this.state` and **lifecycle methods** (`componentDidMount`, `componentDidUpdate`, `componentWillUnmount`). Still found in legacy code and error boundaries.

8. useEffect and Side Effects

8.1 What is an Effect?

An **effect** (`useEffect`) runs code **after render** to synchronise a component with an external system (network, subscriptions, timers, DOM APIs), with optional cleanup.

8.2 Common Examples

Typical uses of effects: subscribing to events, setting up timers, connecting to a chat or WebSocket, and integrating non-React widgets, each with matching cleanup.

8.3 Fetching Data in an Effect (and Race Conditions)

Loading data inside an effect, and preventing **race conditions** (an older, slower response overwriting a newer one) with an ignore flag or an AbortController in the cleanup.

8.4 Dependency Array Pitfalls

Common mistakes with the dependency array: missing dependencies (stale values), objects or functions recreated on every render (infinite loops), and disabling the lint rule.

8.5 You Might Not Need an Effect

Many effects are unnecessary. Derived data should be computed during render, and responses to user actions belong in event handlers rather than in effects.

8.6 useLayoutEffect

`useLayoutEffect` works like useEffect but runs **synchronously after DOM changes and before the browser paints**. Use it to measure layout and avoid visual flicker.

8.7 useEffectEvent (React 19.2)

`useEffectEvent` (React 19.2) defines a function, called from an effect, that always sees the **latest** props and state without needing to be a dependency of the effect.

9. Hooks in Depth

9.1 Rules of Hooks

The two **Rules of Hooks**: call hooks only at the **top level** (not inside conditions or loops), and only from React function components or custom hooks.

9.2 Hooks Overview

Hooks are functions starting with `use` that give function components React features: state, effects, context, refs, memoisation and more.

9.3 useRef

`useRef` returns a mutable `{ current }` object that persists across renders **without causing re-renders**. It is used for DOM access and for storing mutable values.

9.4 useReducer

`useReducer` manages complex state with a **reducer** function `(state, action) => newState` and a `dispatch` function. It suits state with many related transitions.

9.5 useContext

`useContext` reads the current value of a React **context** from the nearest Provider above, without passing props through every level.

9.6 useMemo

`useMemo` **caches the result** of an expensive calculation between renders and recomputes it only when its dependencies change.

9.7 useCallback

`useCallback` **caches a function definition** between renders, so a memoised child receives the same function reference.

9.8 useId

`useId` generates a **unique, stable ID** that matches between server and client, for accessibility attributes such as `htmlFor` and `aria-describedby`.

9.9 useTransition and useDeferredValue (Concurrent Features)

Concurrent features: `useTransition` marks a state update as non-urgent so the UI stays responsive, and `useDeferredValue` defers re-rendering with a value that lags behind.

9.10 useSyncExternalStore

`useSyncExternalStore` subscribes a component to an **external store** (outside React) in a way that is safe with concurrent rendering.

9.11 React 19 Hooks: use, useOptimistic

New React 19 hooks: `use` reads a promise or context (and can be called conditionally), and `useOptimistic` shows an optimistic UI value while an async action is in progress.

9.12 Custom Hooks

A **custom hook** is a function starting with use that combines other hooks to share **stateful logic** (not state itself) between components.

10. Context API and State Management

10.1 The Problem: Prop Drilling

Prop drilling is passing props through many intermediate components that don't use them, only to reach a deeply nested component that does.

10.2 Context API Step by Step

The **Context API** shares a value with a whole subtree: create it with `createContext`, provide it with a Provider, and read it with `useContext` at any depth.

10.3 Types of State

Categories of state: **local UI** state, **shared/global client** state, **server** state (cached API data), **URL** state and **form** state. Each has its own best tool.

10.4 Redux Toolkit (RTK)

Redux Toolkit is the official, simplified way to use Redux: a central store with `createSlice` for reducers and actions, Immer for immutable updates, and RTK Query for data fetching.

10.5 Zustand (Lightweight Alternative)

Zustand is a small, hook-based state management library. You create a store with state and actions and use it in any component without Providers.

10.6 Choosing a Tool

A guide to picking a state tool: local state first, Context for low-frequency shared values, Zustand or Redux for complex global state, and TanStack Query for server data.

11. Routing with React Router

11.1 Client-Side Routing

Client-side routing changes the URL and the displayed view **in the browser**, using the History API, without a full page reload from the server.

11.2 Basic Setup

Setting up **React Router**: wrap the app in a router (`createBrowserRouter` or `BrowserRouter`) and map URL paths to components with routes.

11.3 Params, Search Params and Navigation

Reading dynamic URL segments (`useParams`), query strings (`useSearchParams`), and navigating with `<Link>`, `<NavLink>` or `useNavigate`.

11.4 Nested Routes and Layouts

Nested routes render child routes inside a parent **layout** component at the `<Outlet />` position, so shared UI like headers and sidebars stays in place.

11.5 Protected Routes

Protected routes allow access only when a condition is met (for example, the user is logged in) and otherwise redirect, such as to the login page.

11.6 Data Routers: Loaders and Actions

In React Router data routers, a **loader** fetches data before a route renders (`useLoaderData`) and an **action** handles form submissions for that route.

11.7 Lazy-Loaded Routes

Loading a route's code only when the user visits it, using `React.lazy` or the route `lazy` property, to make the initial bundle smaller.

12. Data Fetching and APIs

12.1 Approaches Compared

A comparison of ways to load data: fetch in an effect, a library such as TanStack Query or SWR, router loaders, and Server Components, with the trade-offs of each.

12.2 axios with Interceptors (JWT Example)

axios is an HTTP client library. **Interceptors** run on every request or response, for example to attach a JWT or to refresh an expired token and retry.

12.3 TanStack Query (React Query)

TanStack Query is a server-state library that handles fetching, **caching**, deduplication, background refetching, retries and mutations through hooks such as useQuery.

12.4 Loading, Error and Empty States

Handling every UI state of a request (loading, error, empty and success) so the user always sees meaningful feedback.

12.5 Suspense for Data

Suspense lets a component "wait" for data. React shows the nearest `<Suspense fallback>` until the data is ready, declaratively.

12.6 Avoiding Waterfalls

A **waterfall** is a chain of requests that run one after another because each one waits for the previous render. Avoid it by fetching in parallel, prefetching or hoisting requests.

13. Performance Optimization

13.1 React.memo

`React.memo` wraps a component so it **skips re-rendering** when its props are shallowly equal to the previous props.

13.2 Avoiding Unnecessary Work

Techniques to reduce wasted renders: keep state local, move state down or lift content up, split components, and use stable props, keys and memoisation where measured.

13.3 Code Splitting and Lazy Loading

Code splitting divides the bundle into smaller chunks that load on demand (`React.lazy` + `import()` + Suspense), so the app starts faster.

13.4 Large Lists: Virtualization

Virtualization (windowing) renders only the list rows that are visible on screen, so lists with thousands of items stay fast. Libraries: react-window, TanStack Virtual.

13.5 Debouncing and Throttling

Debouncing runs a function only after the user stops triggering it for a delay (search boxes). **Throttling** runs it at most once per interval (scroll, resize).

13.6 React Compiler

The **React Compiler** is a build-time tool that automatically memoises components and values, so manual useMemo, useCallback and React.memo are rarely needed.

13.7 Other Performance Checklist

Additional performance practices: production builds, bundle analysis, tree-shaking imports, image optimisation, and using transitions for heavy UIs.

14. Advanced Patterns

14.1 Higher-Order Components (HOC)

A **Higher-Order Component** is a function that takes a component and returns a new component with added behaviour, such as withAuth(Component). It is largely replaced by hooks.

14.2 Render Props

Render props is a pattern where a component receives a **function as a prop** and calls it to decide what to render, sharing its logic with the caller.

14.3 Compound Components

Compound components are a set of components that work together and share implicit state through context, such as `<Tabs><Tabs.List><Tabs.Panel>`.

14.4 Controlled vs Uncontrolled Component APIs
A reusable component API can be **controlled** (the parent owns the state through value/onChange props) or **uncontrolled** (the component manages its own state with a default value), or support both.

14.5 Portals
A **Portal** (`createPortal`) renders children into a different DOM node, such as document.body, while keeping them in the same React tree. Used for modals and tooltips.

14.6 Error Boundaries
An **Error Boundary** is a class component that catches JavaScript errors in its child tree during rendering and shows a fallback UI instead of crashing the whole app.

14.7 Suspense and lazy (Recap)
A recap of `React.lazy` (load a component's code on demand) together with <Suspense> (show a fallback while it loads).

14.8 Container / Presentational and Custom Hooks
Separating **container** components (data and logic) from **presentational** components (just UI). Today the logic usually moves into custom hooks instead.

14.9 Polymorphic and Slot Components
A **polymorphic** component can render as different elements through an as prop. **Slot** components accept named pieces of UI as props for flexible layouts.

15. Styling, TypeScript and Testing

15.1 Styling Options
Ways to style React apps: plain CSS, CSS Modules, utility-first **Tailwind CSS**, CSS-in-JS libraries, and component libraries such as MUI or shadcn/ui.

15.2 TypeScript with React
Using **TypeScript** with React to type props, state, events, refs and hooks, which catches bugs at compile time and improves editor autocompletion.

15.3 Testing with React Testing Library
React Testing Library tests components the way users use them, by querying rendered output by role or text and simulating events, rather than testing implementation details.

16. React 18/19, Server Components and Next.js

16.1 React 18 Recap
React 18 introduced **concurrent rendering**, automatic batching, transitions, Suspense on the server, streaming SSR and the `createRoot` API.

16.2 React 19 Highlights

Key React 19 features: Actions and form actions, useActionState, useOptimistic, the use() API, ref as a prop, document metadata support, and stable Server Components.

16.3 React Server Components (RSC)

Server Components run only on the server. They can access data directly and send rendered UI to the client with **zero client JavaScript**. Interactive parts are marked `'use client'`.

16.4 Hydration

Hydration is React attaching event handlers and state to HTML that was already rendered on the server, making the static page interactive. Mismatches cause hydration errors.

16.5 Next.js App Router Essentials

Core concepts of **Next.js App Router**: file-based routes in the app/ folder, layouts, Server and Client Components, data fetching and caching, Server Actions, and route handlers.

16.6 Accessibility (a11y) Essentials

Building UIs that everyone can use, including keyboard and screen-reader users: semantic HTML, labels, alt text, focus management, ARIA only when needed, and sufficient colour contrast.

16.7 Security Checklist

Key front-end security practices: avoid unsanitised HTML (XSS), validate URLs, keep secrets out of client bundles, enforce authorisation on the server, and use secure cookies.

17. Tricky Interview Questions and Output Puzzles

17.1 Output Prediction Puzzles

Short React snippets where you **predict the exact output or behaviour**, testing your understanding of state snapshots, batching, closures, keys and effects.

17.2 Frequently Asked Conceptual Questions

The React theory questions interviewers ask most often, each with a concise answer you can give in one or two sentences.

17.3 Machine-Coding Questions to Practise

Machine-coding rounds ask you to build a small working UI feature, such as a todo list, autocomplete or pagination, within a time limit. This section lists common ones to practise.

18. Quick Revision Cheat Sheet

18.1 Syntax at a Glance

A compact reference of everyday React syntax (components, props, state, effects, hooks, context, routing) in one place for quick lookup.

18.2 Top 25 Tricky Points

The 25 most important React "gotchas" from this guide, gathered for fast revision before an interview.

18.3 Suggested Study Roadmap

A suggested order of study, from JavaScript foundations to advanced React and Next.js, with practice projects for each stage.

Part 4: C# Data Structures and Algorithms

88 definitions in 17 chapters. Numbers match the sections of the CSharp_DSA_Complete_Guide.pdf guide.

1. Introduction to DSA

1.1 How to Use This Guide

A study guide that explains each data structure and algorithm in the same order: the idea, a diagram, C# code with complexity, then tricky points and interview tips.

1.2 What are Data Structures and Algorithms?

A **data structure** is a way of organising data in memory so it can be accessed and changed efficiently. An **algorithm** is a finite, step-by-step procedure that solves a problem.

1.3 A Problem-Solving Framework (UMPIRE)

UMPIRE is a step-by-step method for solving interview problems: **U**nderstand, **M**atch to a known pattern, **P**lan, **I**mplement, **R**eview, **E**valuate complexity.

1.4 Constraints Tell You the Expected Complexity

The input size limits in a problem show which time complexity is acceptable. For example, n up to 10^5 needs about O(n log n) or better, while n up to 20 allows $O(2^n)$.

1.5 C# Toolkit for DSA

The C# types and APIs used most in DSA problems: arrays, `List`, `Dictionary`, `HashSet`, `Stack`, `Queue`, `PriorityQueue`, `StringBuilder`, sorting and LINQ helpers.

2. Complexity Analysis (Big-O)

2.1 Why Analyse Complexity?

Complexity analysis estimates how an algorithm's running time and memory grow as the input grows. It lets you compare solutions without running them.

2.2 Common Complexity Classes

The standard growth rates from fastest to slowest: O(1), O(log n), O(n), O(n log n), $O(n^2)$, $O(2^n)$, O(n!).

2.3 Rules for Calculating Big-O

Rules for working out Big-O: drop constants, keep only the dominant term, **add** the costs of sequential steps, **multiply** the costs of nested loops, and use different variables for different inputs.

2.4 Space Complexity

Space complexity is the extra memory an algorithm uses as a function of input size, including auxiliary data structures and the recursion call stack.

2.5 Best, Average, Worst and Amortised

Best, **average** and **worst** case describe cost for the most favourable, typical and least favourable inputs. **Amortised** cost is the average per operation over a sequence of operations.

2.6 Complexity of Recursive Algorithms

Finding the cost of recursion from the **recursion tree** (number of calls × work per call) or a recurrence relation, e.g. T(n) = 2T(n/2) + O(n) gives O(n log n).

2.7 Complexity of C# Collection Operations

The Big-O cost of common operations on .NET collections, such as `List.Add` amortised O(1), `Insert` O(n), `Dictionary` lookup O(1) average, and `SortedSet` O(log n).

3. Arrays and Strings

3.1 Arrays

An **array** stores elements of one type in **contiguous memory**, so access by index is O(1). Inserting or deleting in the middle costs O(n) because elements must shift.

3.2 Prefix Sum

A **prefix sum** array stores running totals (`prefix[i] = sum of first i elements`), so the sum of any subarray is answered in O(1) after O(n) preprocessing.

3.3 Two Pointers

Two pointers uses two indices moving through the data (from both ends, or one fast and one slow) to solve pair, partition and in-place problems in O(n).

3.4 Sliding Window

Sliding window keeps a range [left, right] that expands and shrinks over the array or string, solving contiguous subarray or substring problems in O(n) instead of $O(n^2)$.

3.5 Kadane's Algorithm (Maximum Subarray)

Kadane's algorithm finds the maximum-sum contiguous subarray in O(n) by tracking the best sum that ends at each position.

3.6 Strings in C#

String handling for DSA in C#: strings are **immutable**, so use `StringBuilder` or `char[]` for building and modifying, and know the common string, char and frequency-count techniques.

3.7 Matrix Patterns

Techniques for 2-D grids: traversing rows and columns, directions arrays, spiral order, rotation, transposition, and in-place marking.

4. Hashing

4.1 How a Hash Table Works

A **hash table** maps keys to array buckets with a **hash function**, so insert, delete and lookup take O(1) on average. Collisions are handled by chaining or probing.

4.2 Core Patterns

The standard hashing patterns: **frequency counting**, **complement lookup** (Two Sum), **seen-set** for duplicates, **grouping by a key** (anagrams), and **prefix-sum + map**.

4.3 When Hashing Is Not the Answer

Cases where a hash table is the wrong tool: when you need order or range queries, when memory is tight, or when sorting or two pointers already give a simpler solution.

5. Recursion and Backtracking

5.1 Recursion Basics

Recursion is a function calling itself on a smaller sub-problem. It needs a **base case** to stop and a **recursive case** that moves towards it.

5.2 Recursion Tree and Memoization

A **recursion tree** shows every call a recursive function makes. **Memoization** caches the result of each sub-problem so repeated calls are answered instantly.

5.3 Backtracking

Backtracking builds a solution step by step, **choosing** an option, **exploring** further and **undoing** the choice to try others. Used for permutations, subsets and puzzles.

6. Searching and Binary Search

6.1 Linear Search

Linear search checks each element one by one until the target is found. It takes O(n) and works on unsorted data.

6.2 Binary Search

Binary search finds a target in **sorted** data by repeatedly halving the search range, comparing with the middle element each time. It takes O(log n).

6.3 Lower Bound and Upper Bound

Lower bound is the first index with value ≥ target, and **upper bound** is the first index with value > target. Both are binary-search variants for ranges and insertion points.

6.4 Search in Rotated Sorted Array

Binary search on a sorted array that has been rotated. At each step one half is still sorted, which tells you which half can contain the target. O(log n).

6.5 Binary Search on the Answer

Binary search over the **range of possible answers** (not over an array), using a monotonic feasibility check, e.g. minimum capacity or maximum minimum distance.

6.6 Searching a 2-D Matrix

Searching a sorted matrix, either by treating it as one flattened sorted array (O(log mn)) or by a staircase walk from a corner (O(m + n)).

7. Sorting Algorithms

7.1 Overview

Sorting arranges elements in order. Algorithms differ in time and space complexity, **stability** (whether equal items keep their order) and whether they sort in place.

7.2 Bubble, Selection and Insertion Sort

Simple $O(n^2)$ sorts. **Bubble** swaps adjacent out-of-order pairs, **selection** picks the minimum each pass, and **insertion** inserts each item into the sorted prefix (fast on nearly sorted data).

7.3 Merge Sort (Divide and Conquer)

Merge sort splits the array in half, sorts each half recursively and **merges** them. It is O(n log n) in all cases, stable, and needs O(n) extra space.

7.4 Quick Sort

Quick sort picks a **pivot**, partitions elements into smaller and larger sides, and sorts each side recursively. O(n log n) on average, $O(n^2)$ worst case, in place.

7.5 Heap Sort and Counting Sort

Heap sort builds a max-heap and repeatedly extracts the maximum, taking O(n log n) in place. **Counting sort** counts occurrences of small integer keys, taking O(n + k).

7.6 Dutch National Flag (Sort 0s, 1s, 2s)

The **Dutch National Flag** algorithm sorts an array of three values (0s, 1s, 2s) in one pass using three pointers (low, mid, high). O(n) time, O(1) space.

7.7 Custom Sorting in C#

Sorting with custom order in C#: `Array.Sort`/`List.Sort` with a `Comparison` lambda or `IComparer`, and LINQ `OrderBy`/`ThenBy` (stable).

8. Linked Lists

8.1 Structure

A **linked list** is a chain of **nodes**, each holding a value and a reference to the next node (and to the previous one in a doubly linked list). Elements are not stored contiguously.

8.2 Basic Operations
The basic linked-list operations: traversal, insertion and deletion at the head, tail or a given node, and search, often simplified with a **dummy** (sentinel) head node.

8.3 Reverse a Linked List
Reversing the direction of all **next** pointers, iteratively with three pointers (prev, curr, next) or recursively. O(n) time.

8.4 Fast and Slow Pointers (Floyd)
Floyd's tortoise and hare: a slow pointer moves one step and a fast pointer two. Used to detect cycles, find a cycle's start, and find the middle of a list.

8.5 Classic Problems
Standard linked-list interview problems: merge two sorted lists, remove the nth node from the end, palindrome check, intersection of two lists, and reordering.

8.6 LRU Cache (Dictionary + Doubly Linked List)
An **LRU** (Least Recently Used) cache evicts the item unused for the longest time. A dictionary plus a doubly linked list gives O(1) get and put.

9. Stacks and Queues

9.1 Concepts
A **stack** is Last-In-First-Out (push and pop at the top). A **queue** is First-In-First-Out (enqueue at the back, dequeue from the front). Both operations are O(1).

9.2 Stack Problems
Problems solved with a stack: matching brackets, evaluating expressions, undo history, simulating recursion, and min-stack designs.

9.3 Monotonic Stack
A **monotonic stack** keeps its elements in increasing or decreasing order. It solves **next greater/smaller element** and histogram problems in O(n).

9.4 Queue Problems and Deque
Problems solved with a queue (BFS, scheduling, sliding windows) and a **deque**, a double-ended queue, for example for the sliding window maximum.

10. Trees and Binary Search Trees

10.1 Terminology
Tree vocabulary: **root**, **parent/child**, **leaf**, **depth**, **height**, **subtree**, and types such as binary, full, complete, perfect and balanced trees.

10.2 Traversals
Ways to visit every node of a tree: depth-first **pre-order**, **in-order** and **post-order**, and breadth-first **level-order**.

10.3 Classic Recursive Tree Problems
Tree problems solved naturally with recursion: maximum depth, same tree, symmetric tree, path sum, diameter, invert, and lowest common ancestor.

10.4 Binary Search Tree Operations
A **Binary Search Tree** keeps left subtree < node < right subtree, so search, insert and delete take O(h): O(log n) when balanced. In-order traversal gives sorted output.

10.5 Build Trees and Serialize
Reconstructing a tree from traversal arrays (e.g. pre-order + in-order), and **serialization**: converting a tree to a string and back.

11. Heaps and Priority Queues

11.1 What is a Heap?
A **heap** is a complete binary tree where each parent is ≤ its children (min-heap) or ≥ them (max-heap). It is stored in an array and gives O(1) peek and O(log n) insert and remove.

11.2 PriorityQueue in C# (.NET 6+)
`PriorityQueue<TElement, TPriority>` is .NET's built-in **min-heap**. Dequeue returns the lowest priority; use a custom comparer for max-heap behaviour.

11.3 Implementing a Min-Heap
Building a min-heap by hand in an array, with **sift-up** after insert and **sift-down** after removing the root, using index arithmetic (children at 2i+1 and 2i+2).

11.4 Top-K Pattern
Top-K problems (the k largest, smallest or most frequent elements) are solved by keeping a heap of size k, giving O(n log k).

11.5 Merge K Sorted Lists
Merging k sorted lists into one sorted list using a min-heap of the current head of each list. O(N log k).

11.6 Median of a Data Stream (Two Heaps)
Keeping a running median with **two heaps**: a max-heap for the lower half and a min-heap for the upper half, balanced so the median is always at the top.

12. Graphs

12.1 Terminology and Representation

A **graph** is a set of **vertices** connected by **edges**, which may be directed or undirected, weighted or unweighted. It is stored as an **adjacency list** or an **adjacency matrix**.

12.2 Breadth-First Search (BFS)

BFS explores a graph **level by level** using a queue. It finds the shortest path in unweighted graphs. O(V + E).

12.3 Depth-First Search (DFS)

DFS explores as **deep** as possible before backtracking, using recursion or a stack. Used for connectivity, cycle detection, components and topological order. O(V + E).

12.4 Grid Problems (Implicit Graphs)

Treating a 2-D grid as an **implicit graph**, where each cell is a node connected to its neighbours. Used for flood fill, number of islands and shortest path in a maze.

12.5 Cycle Detection

Checking whether a graph has a **cycle**: DFS with recursion-stack colours for directed graphs, and parent tracking or Union-Find for undirected graphs.

12.6 Topological Sort (Kahn's Algorithm)

Topological sort orders the vertices of a directed acyclic graph so every edge goes from earlier to later. **Kahn's algorithm** repeatedly removes nodes with in-degree 0.

12.7 Shortest Paths

Algorithms for the minimum-cost path: **BFS** (unweighted), **Dijkstra** (non-negative weights, using a priority queue) and **Bellman-Ford** (handles negative weights).

12.8 Minimum Spanning Tree (Kruskal)

A **Minimum Spanning Tree** connects all vertices with the smallest total edge weight. **Kruskal's algorithm** adds the cheapest edges in order, using Union-Find to avoid cycles.

13. Tries, Union-Find and Range Query Structures

13.1 Trie (Prefix Tree)

A **Trie** (prefix tree) stores strings character by character along tree paths, so word insert, search and **prefix** queries take O(length of the word).

13.2 Union-Find (Disjoint Set Union)

Union-Find (Disjoint Set Union) tracks which elements are in the same group, with near-O(1) **find** and **union** using path compression and union by rank.

13.3 Fenwick Tree (Binary Indexed Tree)

A **Fenwick Tree** (Binary Indexed Tree) is a compact array structure for **prefix sums with point updates**, both in O(log n).

13.4 Segment Tree

A **Segment Tree** is a binary tree over array ranges that answers range queries (sum, min, max) and handles updates in O(log n).

14. Greedy Algorithms and Intervals

14.1 The Greedy Idea

A **greedy algorithm** makes the **locally best choice** at each step and never revisits it. It gives the right answer only when the problem has the greedy-choice property.

14.2 Classic Greedy Problems

Well-known problems with correct greedy solutions: activity selection, jump game, gas station, minimum coins (for canonical coin systems), and task scheduling.

14.3 Interval Problems

Problems on ranges [start, end] (merge overlapping intervals, insert an interval, count meeting rooms, find the minimum to remove), usually solved by sorting on start or end.

15. Dynamic Programming

15.1 What is DP?

Dynamic Programming solves problems with **overlapping sub-problems** and **optimal substructure** by solving each sub-problem once and storing the result.

15.2 The DP Recipe

A step-by-step method for DP: define the **state**, write the **recurrence**, set the **base cases**, choose top-down (memo) or bottom-up (table), then optimise space.

15.3 1-D DP

DP where the state is a single index, such as climbing stairs, house robber, decode ways and longest increasing subsequence. Often reducible to O(1) space.

15.4 2-D DP

DP with two indices, usually over two strings or a grid: longest common subsequence, edit distance, unique paths, and the 0/1 knapsack.

15.5 Other Common DP Families

Other frequently tested DP families, such as knapsack variants, interval DP, DP on trees, state machines (stock problems) and bitmask DP.

16. Bit Manipulation and Math

16.1 Bitwise Operators

Operators that work on individual bits: AND &, OR |, XOR ^, NOT ~, and the left and right shifts << and >>.

16.2 Essential Tricks

Common bit tricks: check, set or clear the i-th bit, `n & (n-1)` to clear the lowest set bit, power-of-two checks, XOR to find the single number, and counting set bits.

16.3 Math for Coding Interviews

Maths that appears in coding interviews: GCD and LCM, prime checks and the sieve, modular arithmetic, fast exponentiation, and combinatorics basics.

17. Patterns, Interview Strategy and Revision

17.1 Problem-to-Pattern Cheat Sheet

A table that maps problem clues ("sorted array", "top K", "shortest path"...) to the pattern that usually solves them, with its typical complexity.

17.2 Edge Case Checklist

The edge cases to test before submitting: empty input, a single element, duplicates, negative numbers, overflow, already sorted or reversed input, and very large inputs.

17.3 Common C# Pitfalls in Coding Rounds

C#-specific mistakes in coding rounds, such as integer overflow (`int` vs `long`), string concatenation in loops, modifying a collection while iterating, and `PriorityQueue` ordering.

17.4 Must-Practise Problem List (by Topic)

A curated list of must-solve practice problems, grouped by topic, that cover every pattern in this guide.

17.5 Quick Quiz: Name the Pattern and Complexity

A self-test: for each short problem statement, name the pattern that solves it and the resulting time and space complexity.

17.6 Study Roadmap

A suggested week-by-week order of study, from complexity and arrays to graphs and dynamic programming, with practice goals for each stage.